



Semaine 8 : Objets imbriqués

Objectif : Construire des structures riches en mettant des objets DANS des objets **Durée** : 2 heures



Ce que tu vas apprendre

1. Mettre un objet dans un objet
2. Accéder à une propriété en profondeur
3. Modifier une valeur imbriquée
4. Mettre un tableau dans un objet
5. Combiner : tableau d'objets contenant eux-mêmes des objets

1

L'idée des poupées russes

Une propriété d'un objet peut contenir n'importe quoi... **y compris un autre objet !**

```
let heros = {  
  nom: "Steve",  
  pv: 20,  
  stats: {  
    force: 12,  
    agilité: 8,  
    intelligence: 5  
  }  
};
```

Visuellement :

```
heros
├─ nom      : "Steve"
├─ pv       : 20
└─ stats    : {
    │   force      : 12
    │   agilite    : 8
    └─ intelligence : 5
  }
```

2 Accéder en profondeur

On chaîne les **points** :

```
heros.stats.force      // 12
heros.stats.agilite    // 8
```

 **Exercice** : `exercice_1_objet_dans_objet.js`  **Exercice** :
`exercice_2_acceder_profondeur.js`

3 Modifier en profondeur

Pareil, mais à gauche du `=` :

```
heros.stats.force = 15;
heros.stats.intelligence = heros.stats.intelligence + 1;
```

 **Exercice** : `exercice_3_modifier_imbrique.js`

4 Un tableau dans un objet

```
let heros = {  
  nom: "Steve",  
  competences: ["Miner", "Construire", "Combattre"]  
};  
  
console.log(heros.competences[0]); // "Miner"  
heros.competences.push("Cuisiner");
```

Et on peut bien sûr boucler dessus :

```
for (let comp of heros.competences) {  
  console.log("- " + comp);  
}
```

 Exercice : `exercice_4_tableau_dans_objet.js`

5 Combinaison ultime

Un **tableau d'objets**, où chaque objet contient lui-même un **sous-objet** et un **sous-tableau** :

```

let equipe = [
  {
    nom: "Steve",
    stats: { force: 12, agilite: 8 },
    competences: ["Miner", "Combattre"]
  },
  {
    nom: "Alex",
    stats: { force: 9, agilite: 14 },
    competences: ["Tirer à l'arc", "Crafter"]
  }
];

console.log(equipe[0].nom);           // "Steve"
console.log(equipe[1].stats.agilite); // 14
console.log(equipe[0].competences[1]); // "Combattre"

```

 **Exercice :** exercice_5_objet_dans_tableau_dans_objet.js

6 Parcourir une structure complète

```

for (let membre of equipe) {
  console.log(`=== ${membre.nom} ===`);
  console.log(` Force : ${membre.stats.force}`);
  console.log(` Compétences :`);
  for (let comp of membre.competences) {
    console.log(` - ${comp}`);
  }
}

```

 **Exercice :** exercice_6_parcours_complet.js

Pièges à éviter

-  `heros.stats.force` plante si `stats` n'existe pas (`undefined`). → Toujours créer la sous-structure d'abord.
 -  Confondre tableau et objet :
 - `[ ... ]` → on accède par index `tab[0]`
 - `{ ... }` → on accède par clé `obj.cle`
-

Défi de la semaine

 Défi : `defi_equipe_rpg.js`

Crée une équipe de 3 héros, avec stats imbriquées et compétences, puis écris des fonctions pour l'analyser.

À retenir

- Un objet peut contenir n'importe quoi : valeur, objet, tableau
- Accès en chaîne avec les points : `a.b.c.d`
- On combine `for...of` (tableau) et `.propriete` (objet) à volonté